

## Course: CST150: Web technologies

### Lab 4: Deeper Dive into Frontend Responsive Design

## Objective

By the end of this lab, students will:

- Understand advanced responsive design techniques.
- Learn how to use Flexbox alongside CSS Grid.
- Implement a fully responsive web layout with media queries.

As always, don't just follow the workshop verbatim, tweak the changes and see the difference. Share your findings with the group.

## Pre-requisites

- Completion of Lab 3 (JavaScript Fundamentals and Frontend Basics).
- Basic understanding of HTML, CSS, and JavaScript.

## Project Setup

Start from your completed Lab 3 project. Do not create Lab 4 from an empty folder.

1. Duplicate the folder named CST150\_lab3.
2. Rename the duplicated folder to CST150\_lab4.
3. Open CST150\_lab4 in your code editor.
4. Make sure the folder contains index.html, styles.css, and script.js

```
CST150_lab4/  
|-- index.html  
|-- styles.css  
|-- script.js
```

## How Lab 3 and Lab 4 Are Merged

| Part       | Kept from Lab 3                                                  | Added in Lab 4                                                |
|------------|------------------------------------------------------------------|---------------------------------------------------------------|
| HTML       | Button and paragraph for DOM manipulation.                       | Header, nav, main, aside, and footer layout sections.         |
| CSS        | Basic styling can remain if useful.                              | CSS Grid, Flexbox navigation, cards, mobile media queries.    |
| JavaScript | Variables, function, DOMContentLoaded, click event, console log. | Improved message and testing inside the responsive layout.    |
| Testing    | Console output and button click.                                 | Responsive testing using browser resizing and device toolbar. |

### Task 1: Update index.html with a Responsive Page Structure

Replace the content of index.html with the following code. This code keeps the Lab 3 button and paragraph, but places them inside the main content area of the Lab 4 layout.

**Important:** The script.js link must stay before the closing `</body>` tag so the JavaScript can find the HTML elements after the page loads.

### What Students Should Notice in the HTML

- The header introduces the page.
- The nav contains links and will be styled using Flexbox.
- The main section contains the Lab 3 JavaScript activity.
- The aside works like a sidebar for extra information.
- The footer closes the page with course information.
- The id values `changeText` and `text` are important because script.js uses them.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Lab 4 Responsive Frontend with JavaScript</title>
  <link rel="stylesheet" href="styles.css" />
</head>
<body>
  <header>
    <h1>Responsive Frontend Development</h1>
    <p>Combining Lab 3 JavaScript with Lab 4 Responsive Design</p>
  </header>
  <nav>
    <ul>
      <li><a href="#">Home</a></li>
      <li><a href="#">Activities</a></li>
      <li><a href="#">Gallery</a></li>
      <li><a href="#">Contact</a></li>
    </ul>
  </nav>
  <main>
    <section class="card">
      <h2>Lab 3 JavaScript Activity</h2>
      <p id="text">This text will change when you click the button.</p>
      <button id="changeText">Click Me</button>
    </section>

    <section class="card">
      <h2>Real-Life Example</h2>
      <p>
        Imagine this is a student portfolio website. The layout changes
        depending on whether the visitor is using a laptop, tablet, or phone.
      </p>
    </section>
  </main>
  <aside>
    <h2>Sidebar</h2>
    <p>Useful links, announcements, or extra course notes can go here.</p>
  </aside>
  <footer>
    <p>&copy; 2024 CST150 Web Technologies</p>
  </footer>
  <script src="script.js"></script>
</body>
</html>
```

## **Task 2: Update styles.css with CSS Grid, Flexbox, and Media Queries**

Replace the content of styles.css with the following code. The body uses CSS Grid for the full page layout. The navigation uses Flexbox. Media queries change the layout for smaller screens.

### **What Students Should Notice in the CSS**

- CSS Grid controls the big page sections.
- Flexbox controls the navigation links inside the nav area.
- At 768px or below, the layout changes from two columns to one column.
- At 480px or below, the button becomes full width for easier mobile tapping.
- The JavaScript does not change when the screen size changes; it still works because the HTML ids stay the same.

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  display: grid;
  grid-template-areas:
    "header header"
    "nav main"
    "nav aside"
    "footer footer";
  grid-template-columns: 1fr 3fr;
  grid-template-rows: auto 1fr auto auto;
  min-height: 100vh;
  background-color: #f4f4f4;
}

header {
  grid-area: header;
  background-color: #1e3a5f;
  color: white;
  padding: 20px;
  text-align: center;
}

nav {
  grid-area: nav;
  background-color: #2f5d8c;
  padding: 20px;
}

nav ul {
  display: flex;
  flex-direction: column;
  gap: 15px;
  padding: 0;
  margin: 0;
  list-style: none;
}

nav a {
  display: block;
  text-decoration: none;
  color: white;
  padding: 10px;
  background-color: #24496e;
  border-radius: 6px;
  text-align: center;
}
```

```
nav a:hover {
  background-color: #18324d;
}

main {
  grid-area: main;
  padding: 20px;
}

aside {
  grid-area: aside;
  background-color: #e2e8f0;
  padding: 20px;
}

footer {
  grid-area: footer;
  background-color: #1e3a5f;
  color: white;
  text-align: center;
  padding: 15px;
}

.card {
  background-color: white;
  padding: 20px;
  margin-bottom: 20px;
  border-radius: 8px;
  box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
}

button {
  background-color: #2f5d8c;
  color: white;
  border: none;
  padding: 12px 18px;
  border-radius: 6px;
  cursor: pointer;
}

button:hover {
  background-color: #1e3a5f;
}

@media (max-width: 768px) {
  body {
    grid-template-areas:
      "header"
      "nav"
```

```
    "main"  
    "aside"  
    "footer";  
    grid-template-columns: 1fr;  
}  
nav ul {  
    flex-direction: row;  
    flex-wrap: wrap;  
    justify-content: center;  
}  
  
nav a {  
    min-width: 100px;  
}  
}  
@media (max-width: 480px) {  
    header h1 {  
        font-size: 24px;  
    }  
    nav ul {  
        flex-direction: column;  
    }  
    main,  
    aside {  
        padding: 15px;  
    }  
    button {  
        width: 100%;  
    }  
}
```

### Task 3: Keep and Improve the Lab 3 script.js Activity

Update script.js with the following code. This keeps the Lab 3 JavaScript concepts: variable, function, DOMContentLoaded, DOM selection, event listener, style change, and console logging.

```
// Variables and Data Types
let message = "Hello, JavaScript!";
console.log(message);

// Function Declaration
function greetUser() {
    alert("Welcome to JavaScript Fundamentals and Responsive Design!");
}

// Run after the page has loaded
document.addEventListener("DOMContentLoaded", function () {
    greetUser();

    // DOM Manipulation
    let button = document.getElementById("changeText");
    let text = document.getElementById("text");

    button.addEventListener("click", function () {
        text.textContent =
            "You clicked the button! JavaScript is working inside a responsive layout.";
        text.style.color = "blue";

        console.log("Button clicked successfully.");
    });
});
```

**Teaching point:** This shows students that CSS and JavaScript solve different problems. CSS changes the layout. JavaScript changes the behaviour.

### Task 4: Testing the Responsive Design

1. Open [index.html](#) in a browser.
2. Resize the browser window to test responsiveness.
3. Test on different devices or use browser developer tools (F12 → Toggle Device Toolbar).



## Discussion Questions

1. How does combining CSS Grid and Flexbox improve layouts?
2. What advantages do media queries offer for mobile design?
3. How would you make the navigation bar collapse into a dropdown menu for small screens?

## Bonus Challenge

- Implement a responsive image gallery using Flexbox.
- Create a collapsible sidebar for mobile screens.

## Conclusion

In this lab, students deepened their understanding of responsive design by implementing CSS Grid, Flexbox, and media queries. Future lessons will focus on dynamic elements and JavaScript-driven interactivity.

**End of Lab 4**